

LAB MID-TERM PERPARTION

EMBEDDED SYSTEMS

Experiment # 01

Introduction to Edsim51 simulator with demo examples

TASK #1:

This program displays the binary pattern from 0 to 255 (and back to 0) on the LEDs interfaced with port 1. A 1 in the pattern is represented by the LED on, while a 0 in the pattern is represented by the LED off. However, logic 0 on a port 1 pin turns on the LED, therefore it is necessary to write the inverse of the pattern to the LEDs. The easiest way to do this is to send the data FFH to 0 (and back to FFH) to the LEDs. Since port 1 is initially at FFH all we need to do is continuously decrement port 1.

CODE:

start:

DEC P1;decrement port

JMP start;and repeat

OUTPUT:

The screenshot displays the Proteus 8051 simulator interface. The top window shows the 8051 register and memory view. The bottom window shows the hardware components including a keyboard, a DAC, and a 7-segment display.

Register and Memory View:

- System Clock (MHz):** 12.0
- SBUS:** R/O, W/O, TH0, TL0, R7, B, R6, ACC, R5, PSW, R4, IP, R3, IE, R2, PCON, R1, DPH, R0, DPL, SP.
- pins bits:** P3, P2, P1, P0.
- PC:** 0x0002
- PSW:** 00000000
- Modify RAM:** addr 0x00, value 0x00
- Data Memory:** 0-70 hex addresses, each with 16 bits.

Hardware Components:

- DI / LD:** Input/Output pins.
- AND Gate Disabled:** Control for the AND gate.
- Key Bounce Disabled:** Control for key bounce.
- Standard:** Keyboard type.
- U:** UART module.
- No Parity:** Parity control.
- 8-bit UART @ 4800 Baud:** UART configuration.
- Rx Reset:** Reset for the receiver.
- Tx Send:** Send for the transmitter.
- 0.0 V output:** DAC output.
- Scope:** Oscilloscope.
- DAC:** Digital-to-Analog Converter.
- 7-segment display:** Display showing the output of the DAC.

TASK #2:

This program very simply echoes the switches on P2 to the LEDs on P1. When a switch is closed a logic 0 appears on that P2 pin, which is then copied to that P1 bit which turns on that LED. Therefore, a closed switch is seen as a lit LED and vice versa.

CODE:

start:

MOV P1,P2;move data on P2 pins to P1

JMP start;and repeat

OUTPUT:

The screenshot displays the 8051 simulator interface. The top section shows the System Clock (MHz) set to 12.0 and the instruction counter at 60. The register window lists R0-R7, ACC, PSW, IP, IE, PCON, DPH, DPL, and SP. The PC is 8051. The Data Memory window shows a table of memory addresses from 00 to 70, all containing 00. The bottom section shows the I/O devices: a keyboard, a DAC, a scope, and a UART. The UART is configured for 8-bit, No Parity, 4800 Baud. The LCD display shows the text "8888".

TASK #3:

Interface an Analog-to-Digital Converter (ADC) with the 8051 to read an analog signal and display its digital equivalent.

CODE:

```
ORG 0000H
```

```
MOV P1,#0FFH; Set Port 1 as input port (for ADC data)
```

```
MOV P2,#00H; Set Port 2 as output port (for control signals)
```

```
MOV DPTR,#LCD_CMD; Initialize LCD Command Pointer
```

CALL INIT_LCD;Initialize the LCD

START:

SETB P2.0;SEND HIGH TO START CONVERSION(SC PIN)

CLR P2.0;SEND LOW TO STOP CONVERSION

JB P2.1,START;WAIT UNIT END OF CONVERSION(EOC PIN)

MOV A,P1;READ DIGITAL VALUE FROM ADC(8-BIT)

CALL DISPLAY;SEND VALUE TO LCD DISPLAY

SJMP START;REPEAT FOREVER

DISPLAY:

;Code to display the value on LCD (implement your own LCD routine)

RET

INIT_LCD:

; Code to initialize LCD (implement your own LCD routine)

RET

END

OUTPUT:

The screenshot displays the MTS-51 Trainer board software interface. The top section shows the 8051 microcontroller register window with the PC register highlighted at address 0000A. The assembly code editor on the right contains the following code:

```

ORG 0000H
MOV P1, #0FFH; Set Port 1 as input
MOV P2, #00H; Set Port 2 as output
CALL INIT_LCD; Initialize the LCD
START:
SETB P2.0; SEND HIGH TO START CONVE
CLR P2.0; SEND LOW TO STOP CONVERSI
JB P2.1, START; WAIT UNIT END OF CON
MOV A, P1; READ DIGITAL VALUE FROM P
CALL DISPLAY; SEND VALUE TO LCD DI
SJMP START; REPEAT FOREVER
DISPLAY:
; Code to display the value on LCD
RET
INIT_LCD:
; Code to initialize LCD (impleme
RET
END

```

The bottom section shows the hardware simulation panel with a 7-segment display showing '8.8.8.8', a DAC output of 0.0V, and various control buttons like 'AND Gate Disabled', 'Key Bounce Disabled', and 'Standard'.

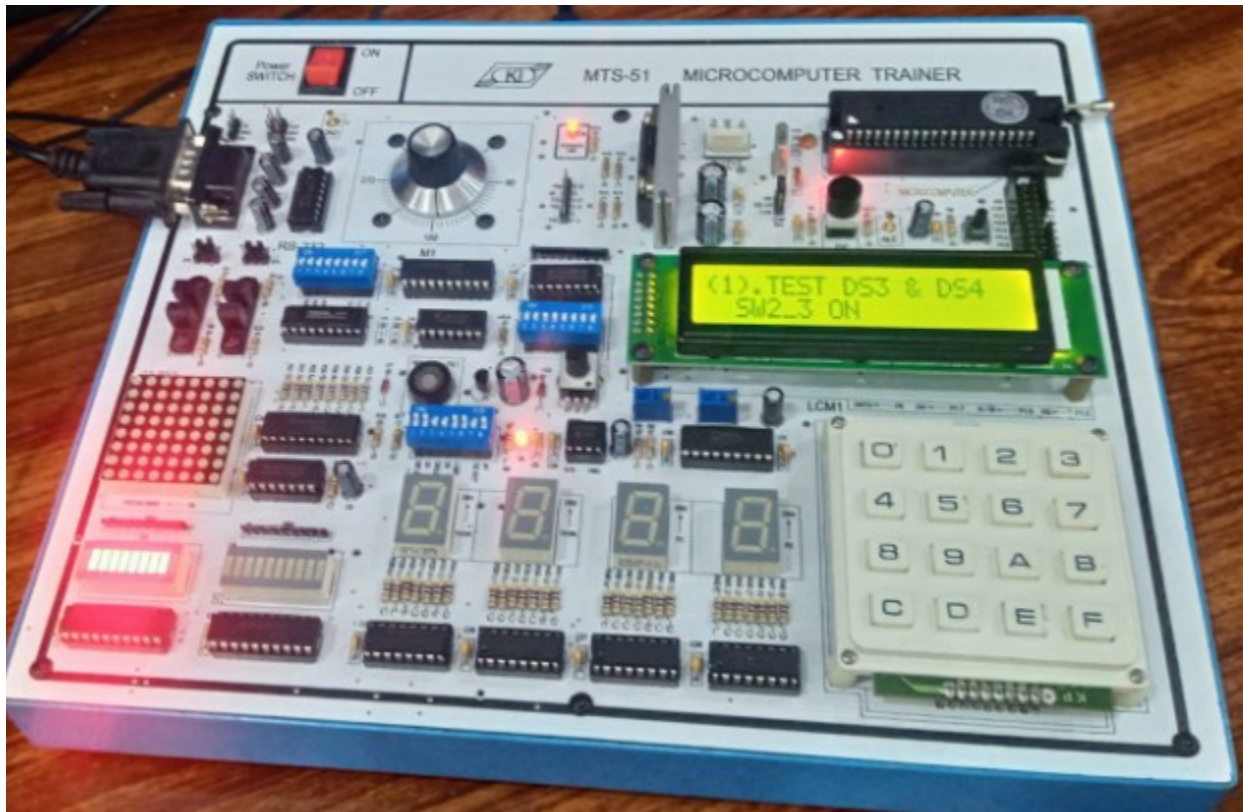
Experiment # 02

Introduction to MTS-51 Trainer board Inside's with demo examples

TASK #1:

Show the seven-segment display 3 and 4 by pressing 1. Write down the steps to have a successful test. Take the snapshot of the final output generated by test.

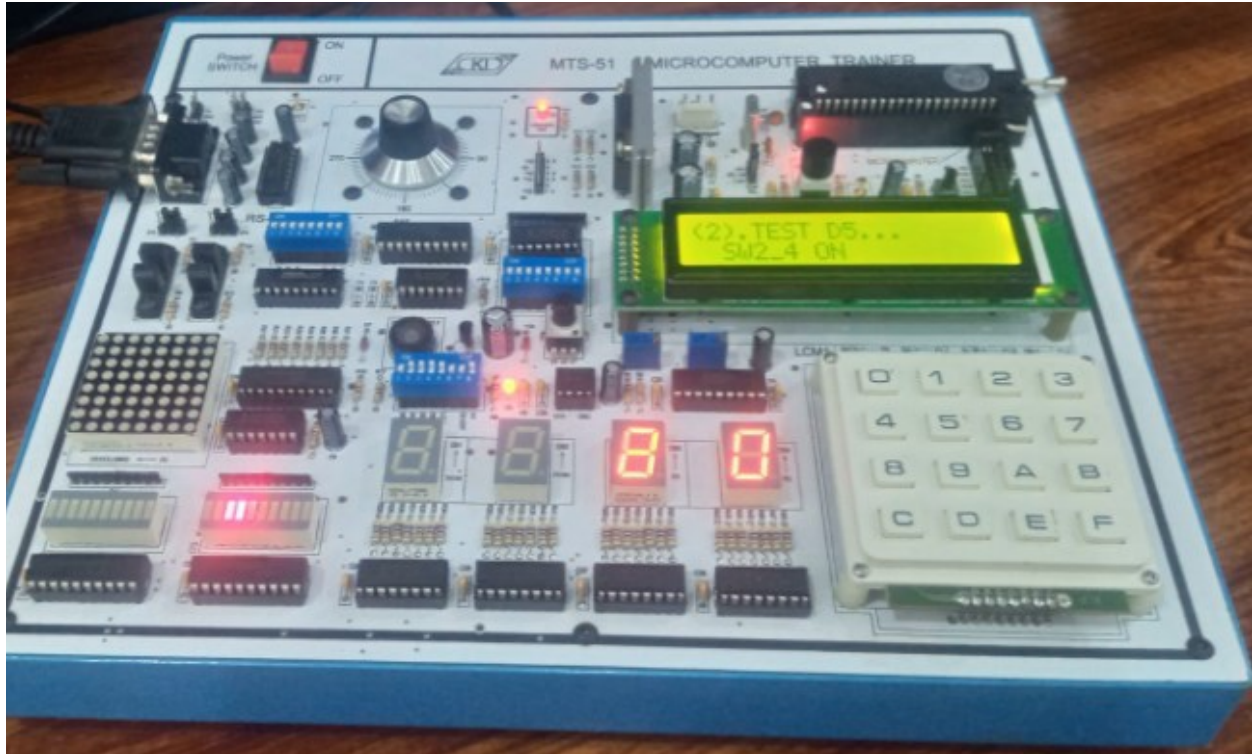
OUTPUT:



TASK #2:

Show the led bar 2 by pressing 2. Write down the steps to have a successful test. Take the snapshot of the final output generated by test.

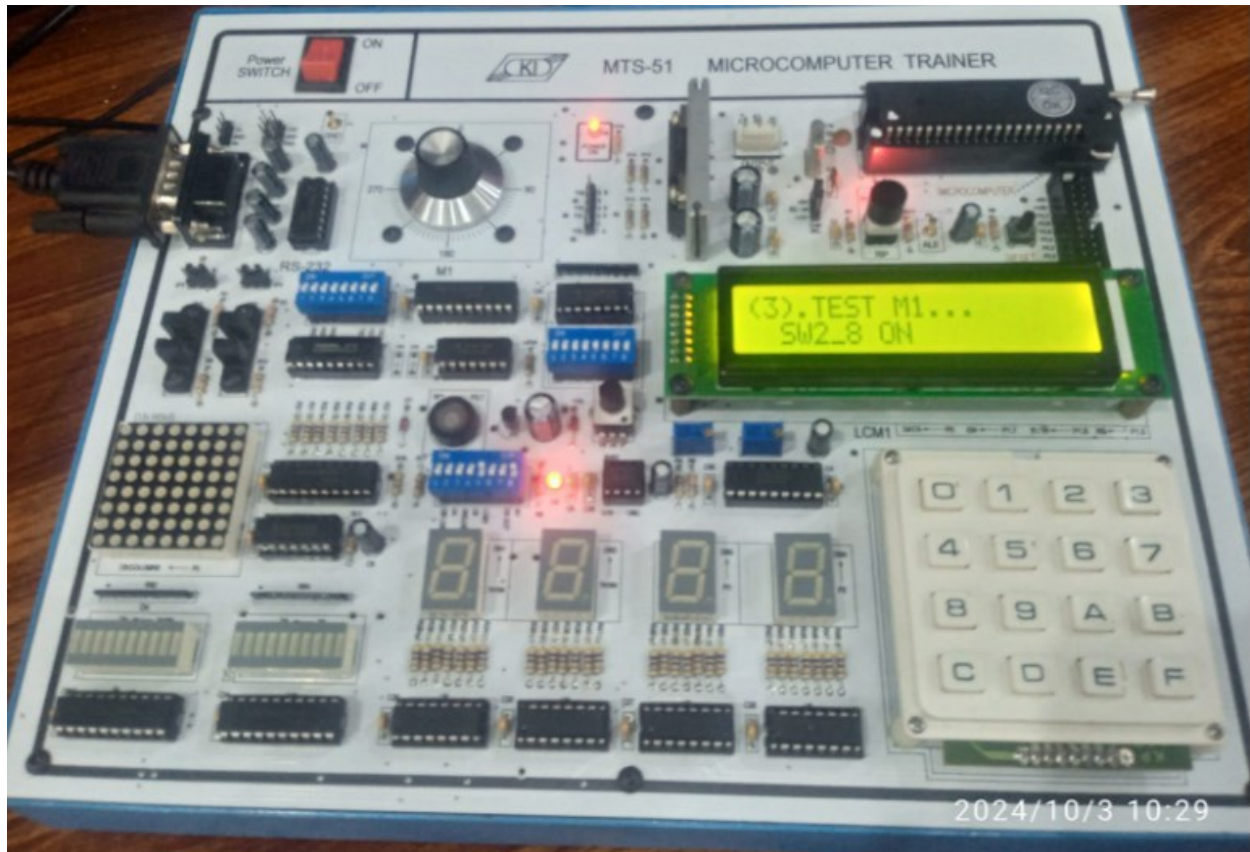
OUTPUT:



TASK #3:

Show the stepping motor by pressing 3. Write down the steps to have a successful test. Take the snapshot of the final output generated by test.

OUTPUT:



TASK #4:

Show the IC 555 timer to speaker by pressing 4. Write down the steps to have a successful test. Take the snapshot of the final output generated by test.

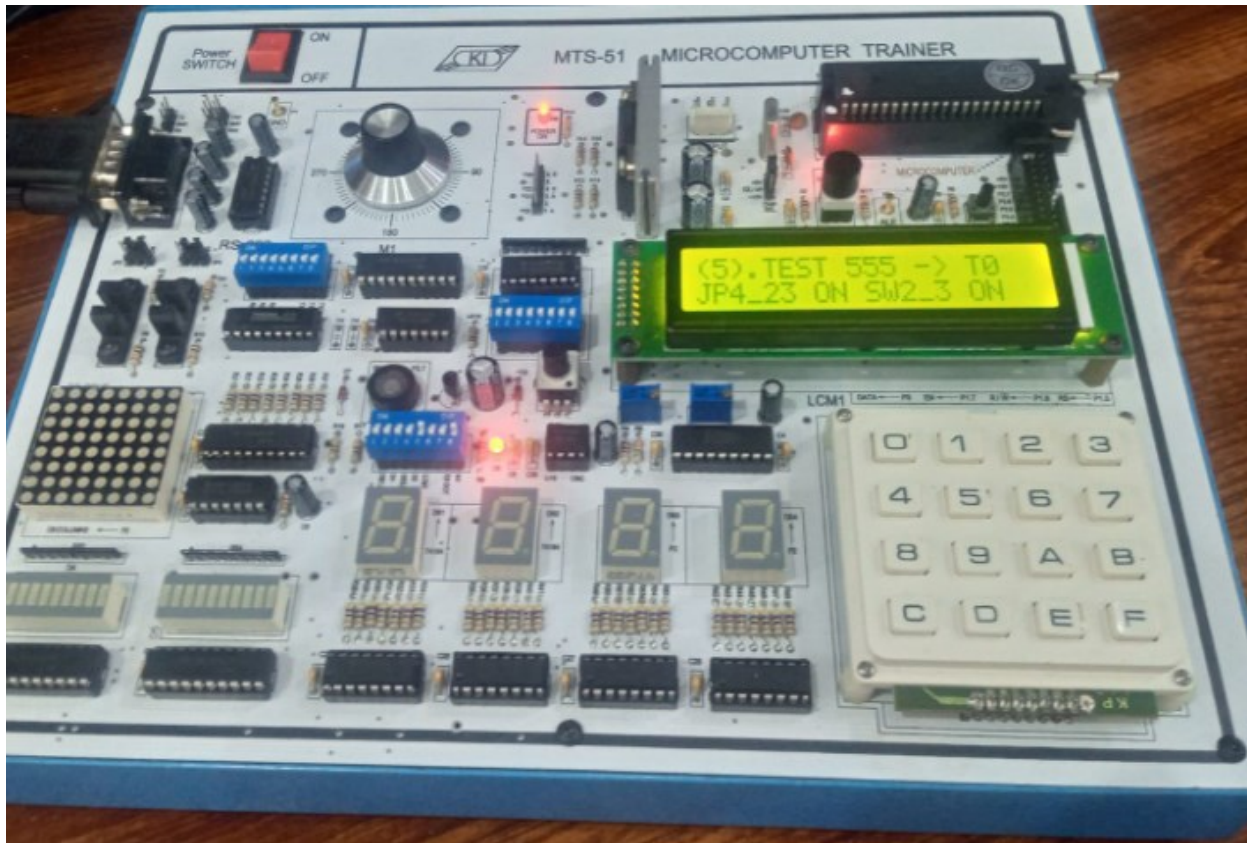
OUTPUT:



TASK #5:

Show the IC 555 timer to seven segment display by pressing 5. Write down the steps to have a successful test. Take the snapshot of the final output generated by test.

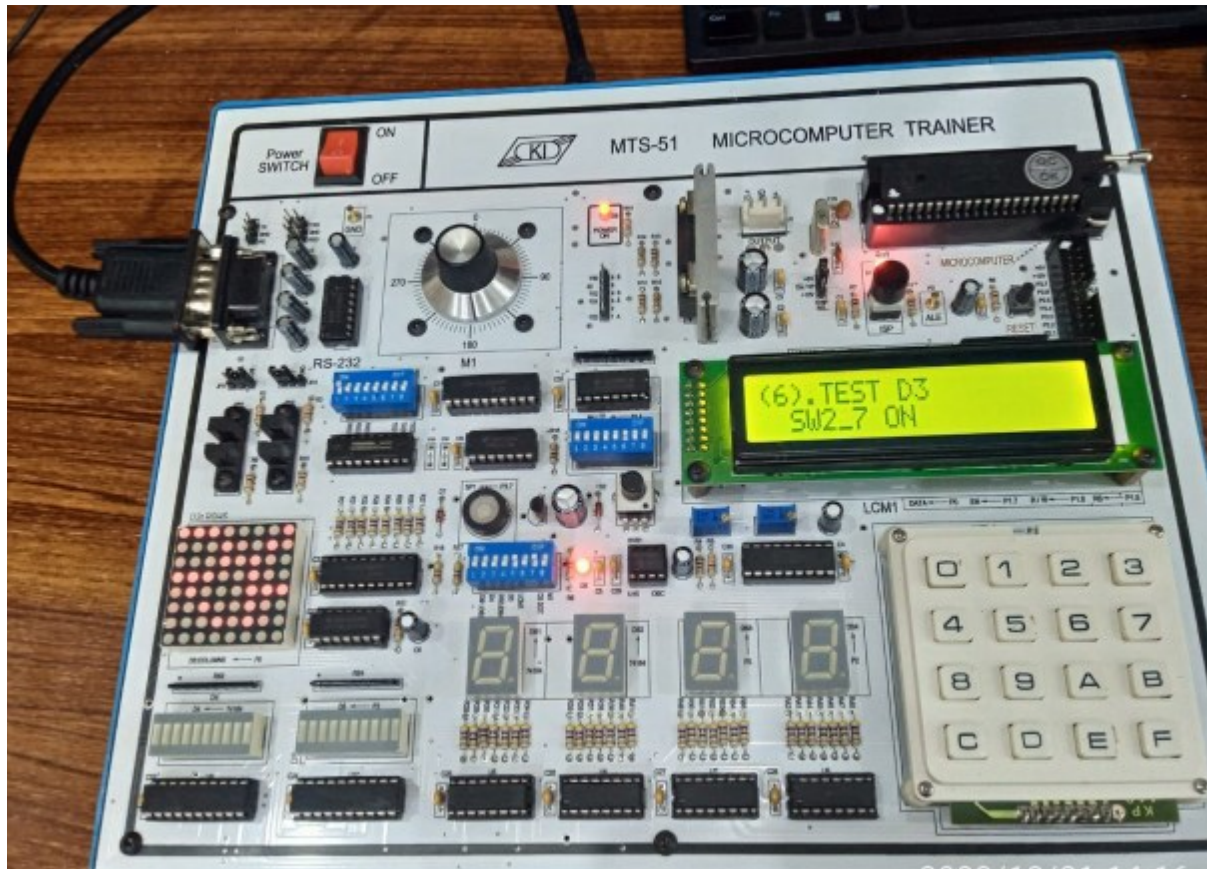
OUTPUT:



TASK #6:

Show the 8x8 dot matrix by pressing 6. Write down the steps to have a successful test. Take the snapshot of the final output generated by test.

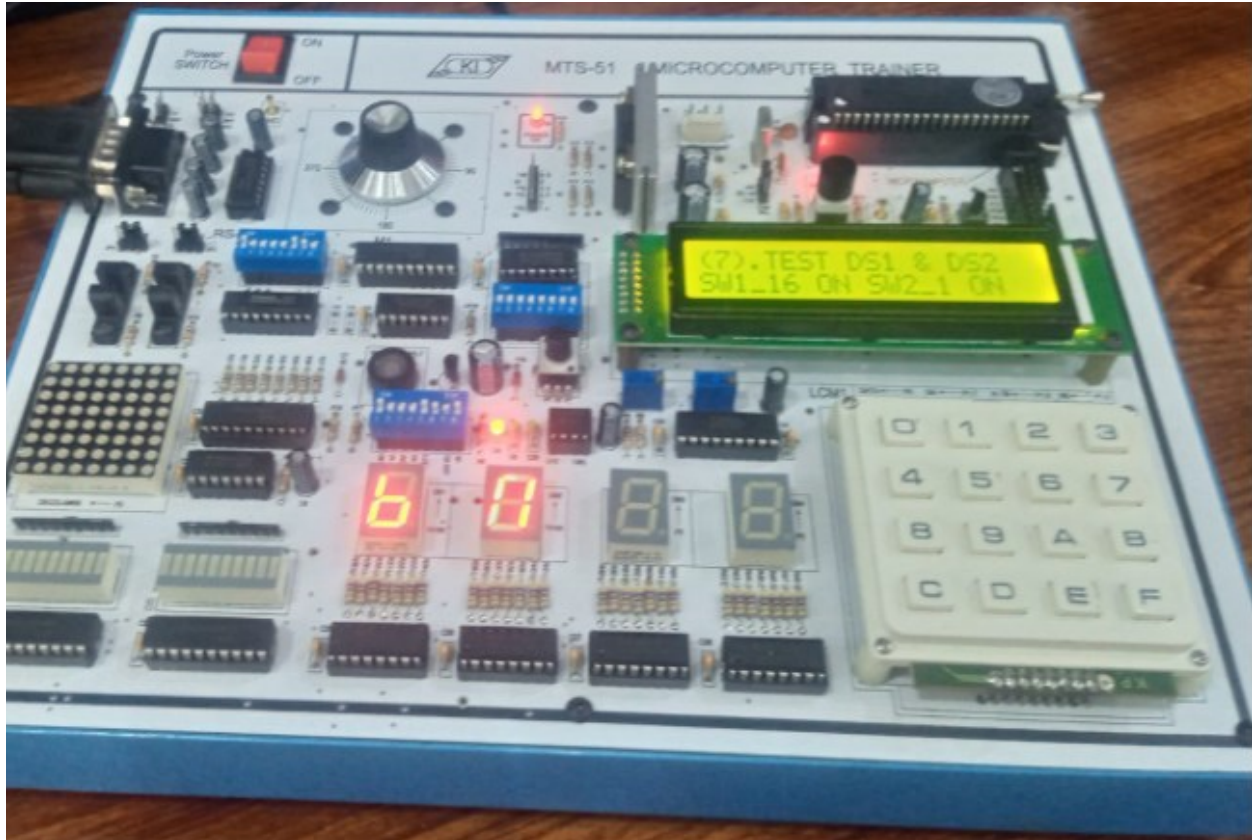
OUTPUT:



TASK #7:

Show the seven-segment display 1 and 2 by pressing 7. Write down the steps to have a successful test. Take the snapshot of the final output generated by test.

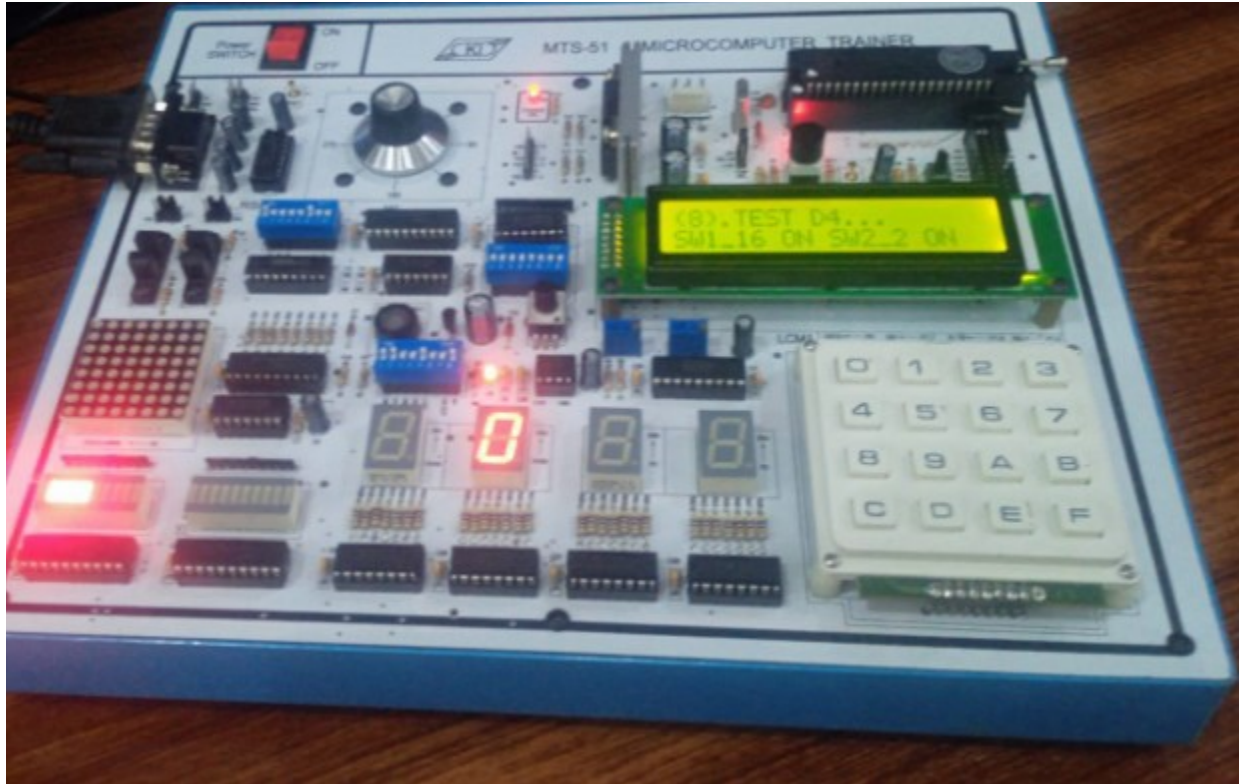
OUTPUT:



TASK #8:

Show the led bar 1 by pressing 8. Write down the steps to have a successful test. Take the snapshot of the final output generated by test.

OUTPUT:



TASK #9:

Show the photo interrupter (PH1) by pressing 9. Write down the steps to have a successful test. Take the snapshot of the final output generated by test.

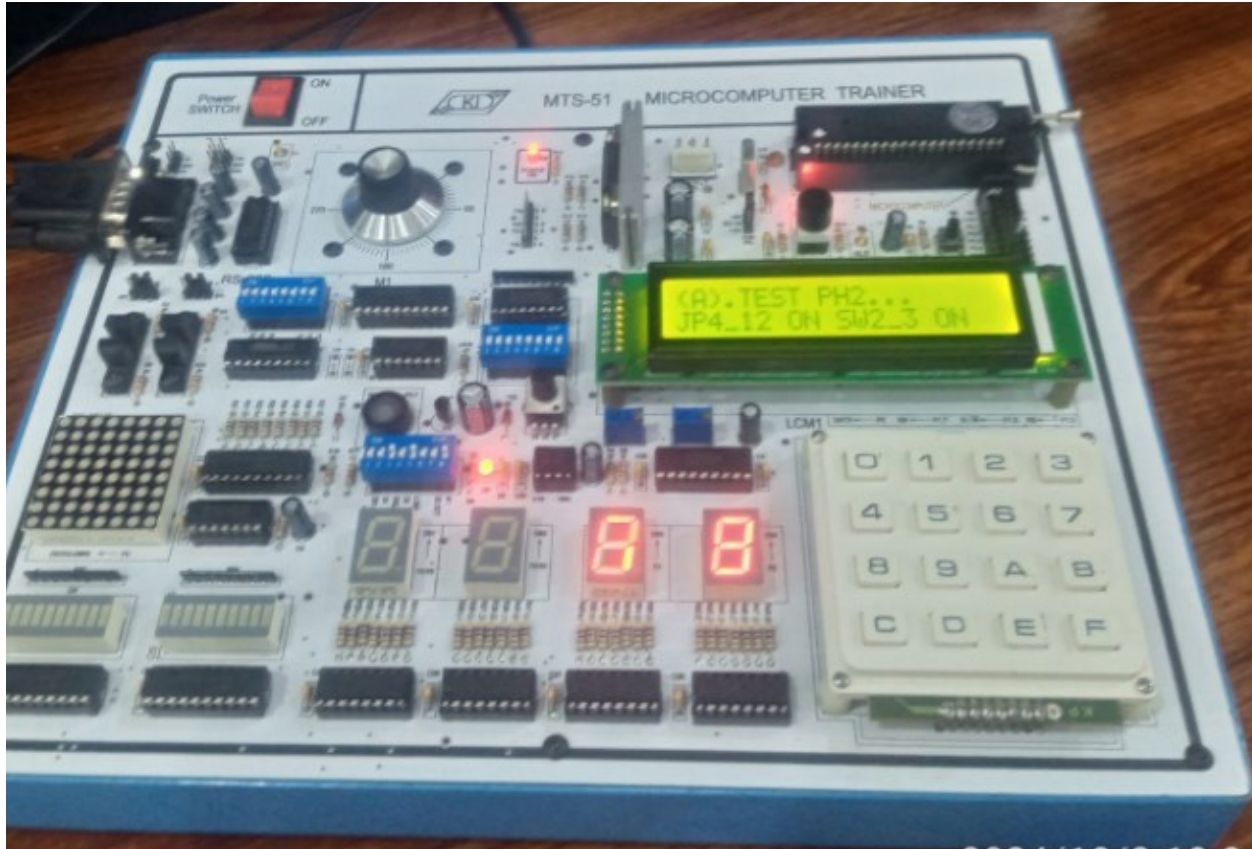
OUTPUT:



TASK #10:

Show the photo interrupter (PH2) by pressing 10. Write down the steps to have a successful test. Take the snapshot of the final output generated by test.

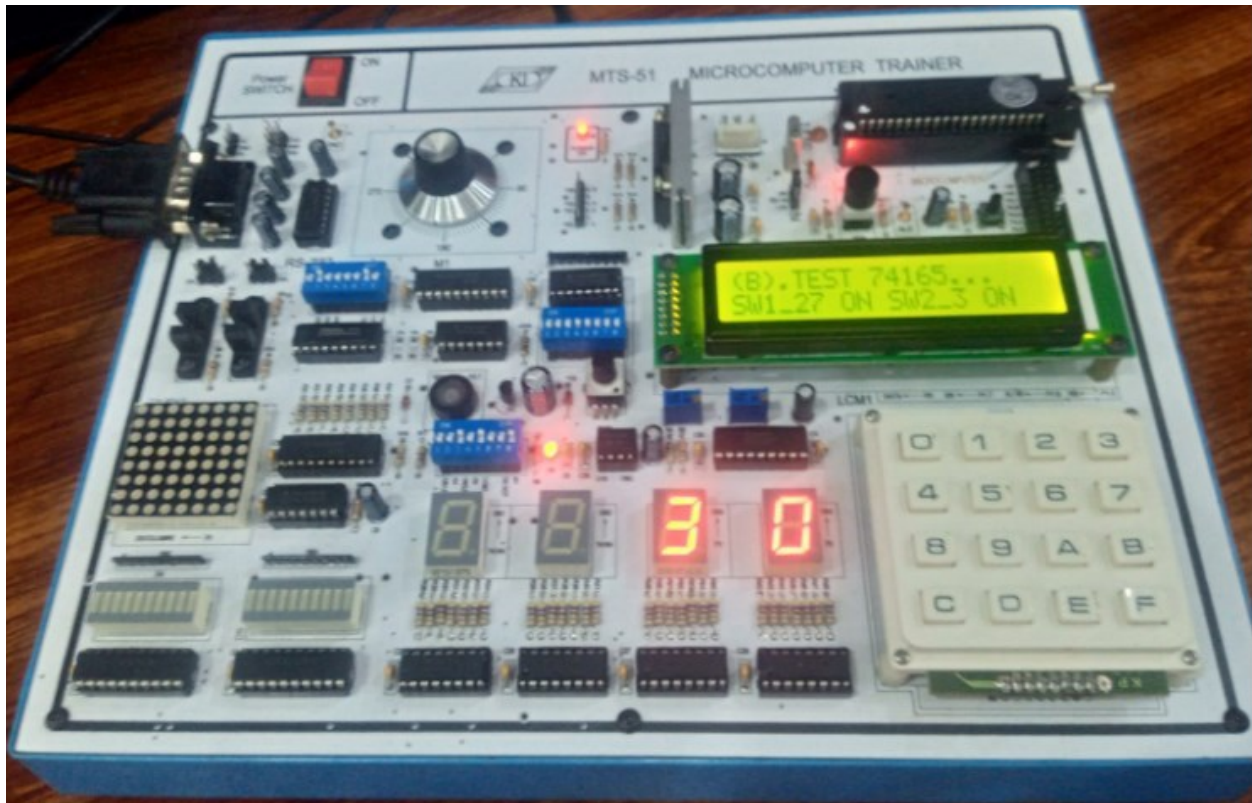
OUTPUT:



TASK #11:

Show the parallel in serial out shift register by pressing 11. Write down the steps to have a successful test. Take the snapshot of the final output generated by test.

OUTPUT:



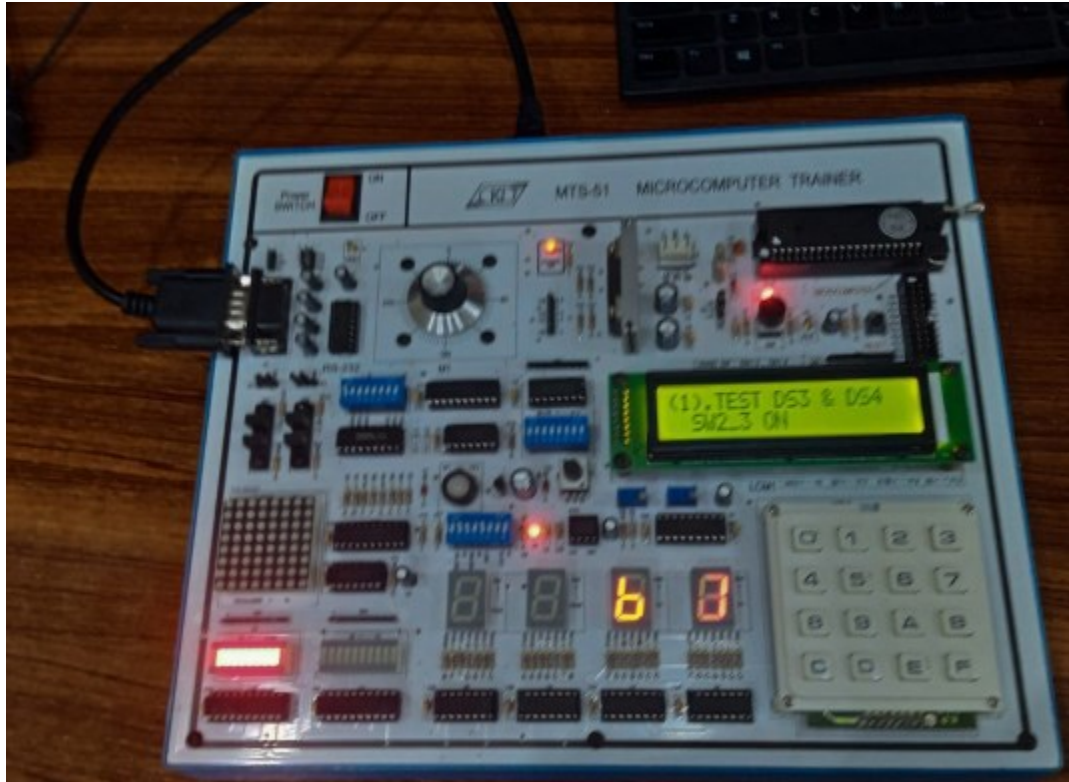
Experiment #3

Working of Flash Magic software with demo examples

TASK #1:

Demo example 1 for LED display control is demonstrated by configuring switches as in the previous lab. The steps for successful run are documented, and a snapshot of the final output is taken.

OUTPUT:



TASK #2:

Show the output of demo example 1 for 7 SEGMENT DISPLAY CONTROL (DS3 AND DS4) by configuring switches as you done in previous lab. Write down the steps to have a successful run. Take the snapshot of the final output generated by example.

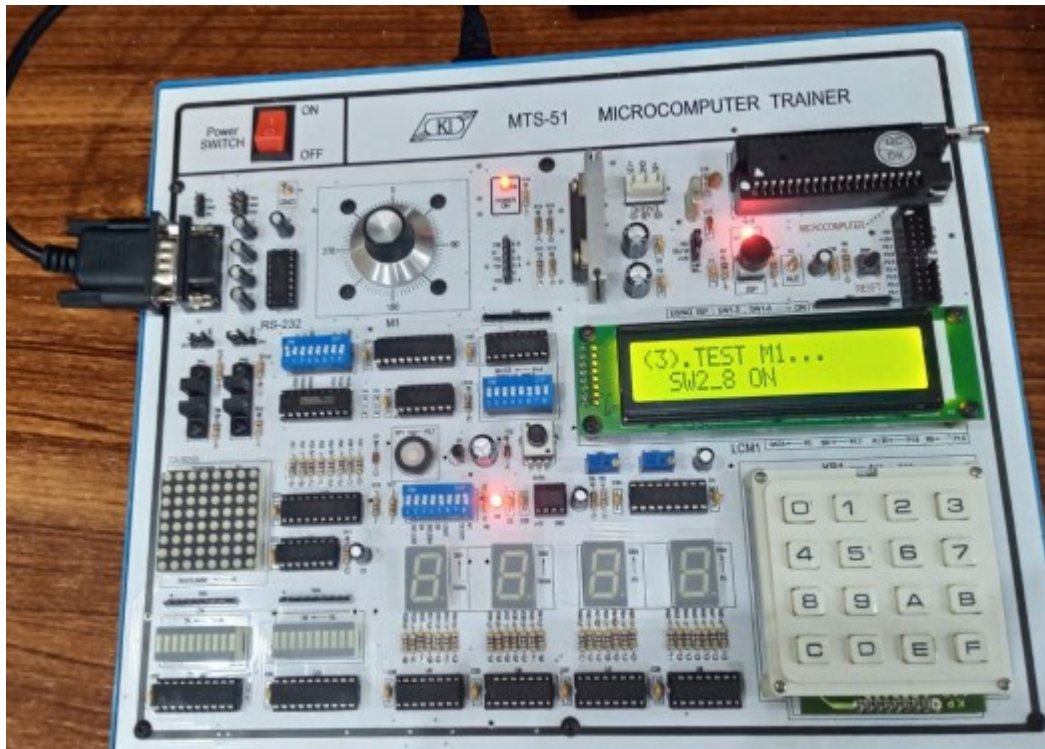
OUTPUT:



TASK #3:

Demo example 1 for DOT MATRIX LED CONTROL is demonstrated by configuring switches as in the previous lab, and the steps for a successful run are documented, along with a snapshot of the final output.

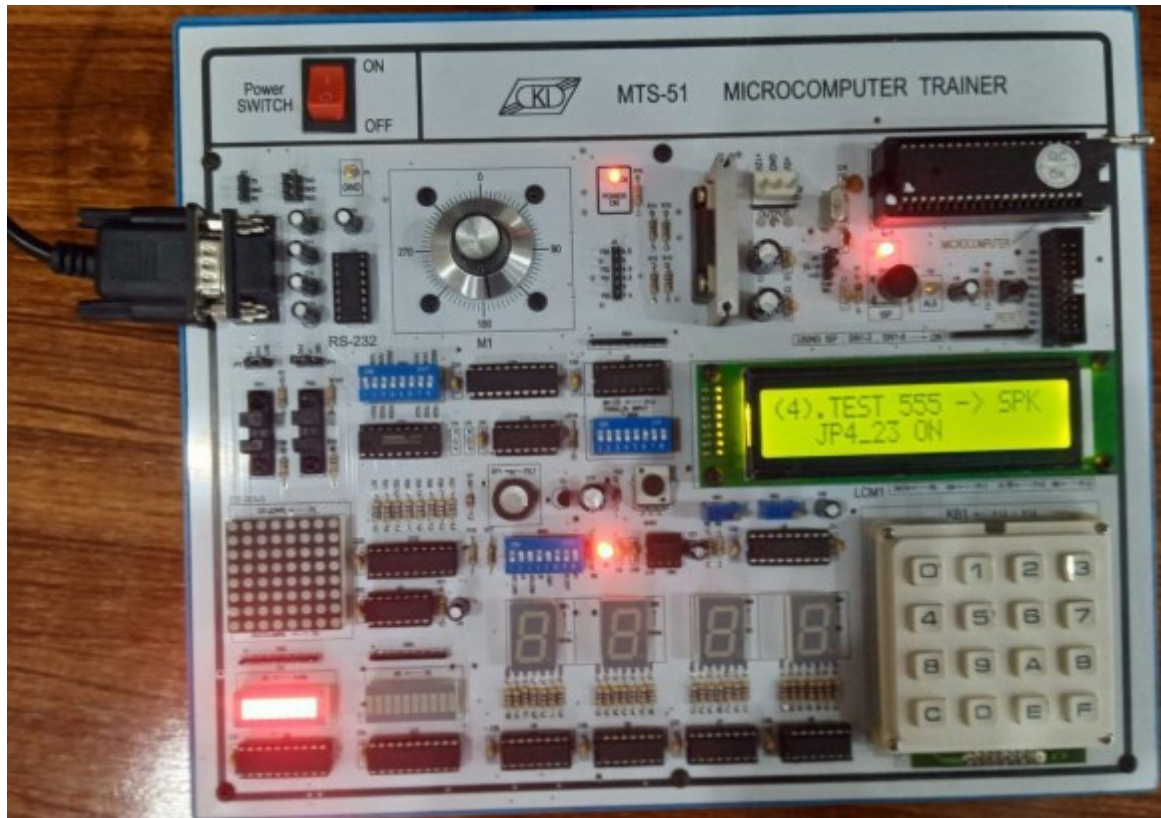
OUTPUT:



TASK #4:

Demo example 1 for MATRIX KEYBOARD CONTROL on DS4 is demonstrated by configuring switches as in the previous lab, and the steps for successful run are documented, along with a snapshot of the final output.

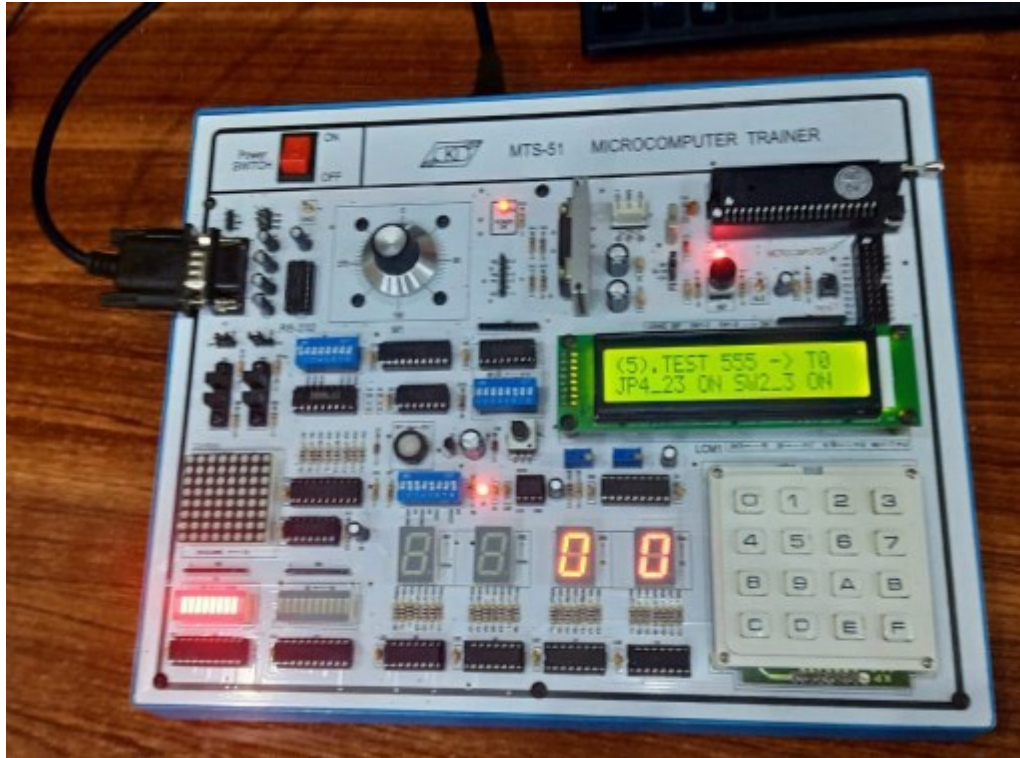
OUTPUT:



TASK #5:

Demo example 1 for STEP MOTOR CONTROL is demonstrated by configuring switches as in the previous lab, with steps for successful run recorded and a snapshot of the final output taken.

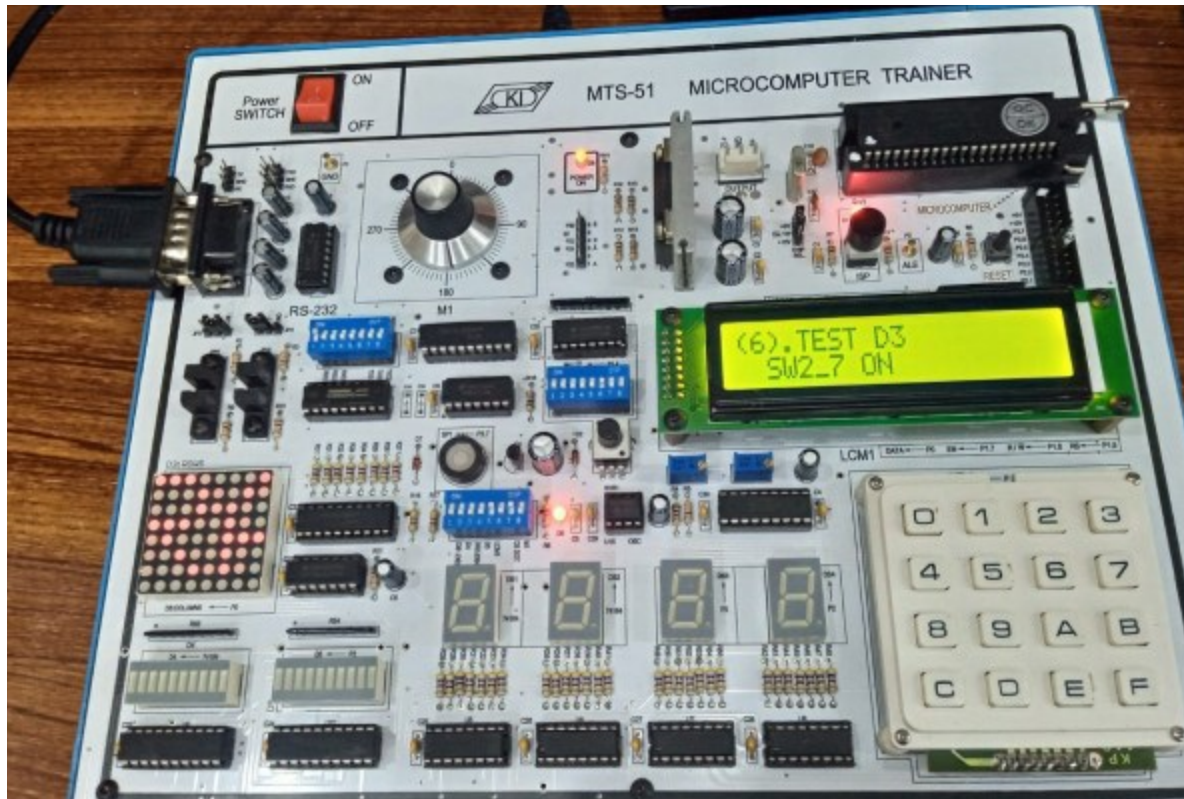
OUTPUT:



TASK #6:

Demo example 1 for PHOTO INTERRUPTER CONTROL (JP4=PH2 >T0) on STEP MOTOR is demonstrated, followed by configuring switches as in the previous lab, and a snapshot of the final output generated.

OUTPUT:



TASK #7:

Demo example 1 for Pulse Counter (JP4=T0->OSC) on DS3 and DS4 is demonstrated, with switches configured as in the previous lab. A snapshot of the final output is taken.

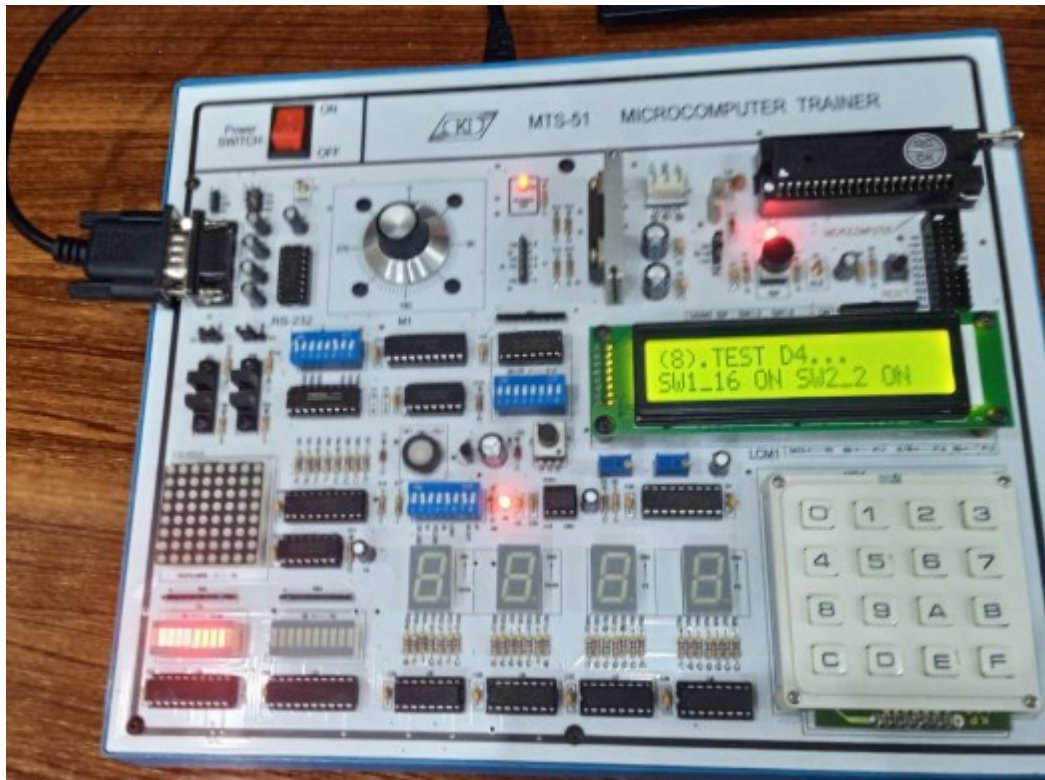
OUTPUT:



TASK #8:

Demo example 1 for TIMER/COUNTER CONTROL on LED BAR is demonstrated by configuring switches as in the previous lab, with steps for successful run and a snapshot of the final output.

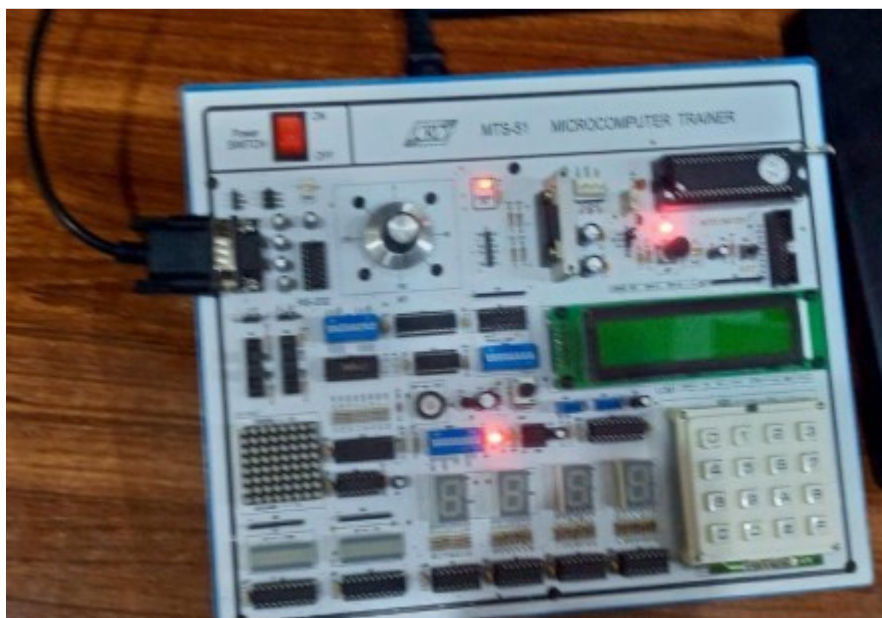
OUTPUT:



TASK #9:

Demo example 1 for SPEAKER CONTROL output is displayed by configuring switches as in the previous lab, with steps for successful run documented and a snapshot taken of the final output.

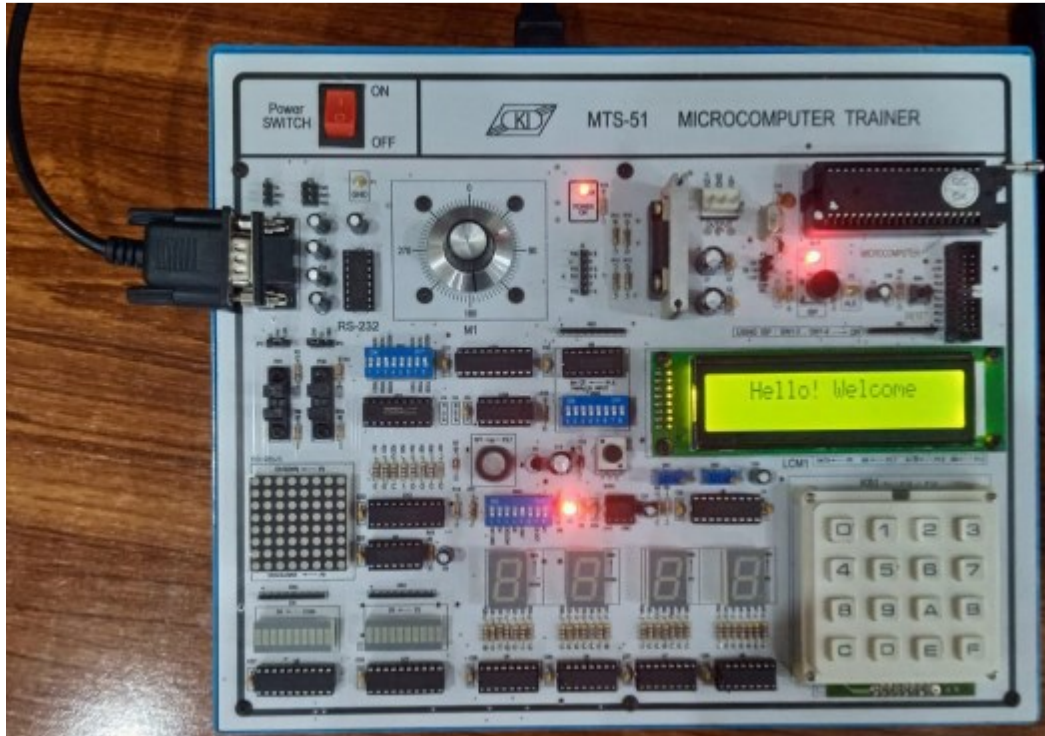
OUTPUT:



TASK #10:

The task involves displaying the output of demo example 1 for LCM DISPLAY CONTROL, configuring switches as in the previous lab, recording the successful run steps, and taking a snapshot of the final output.

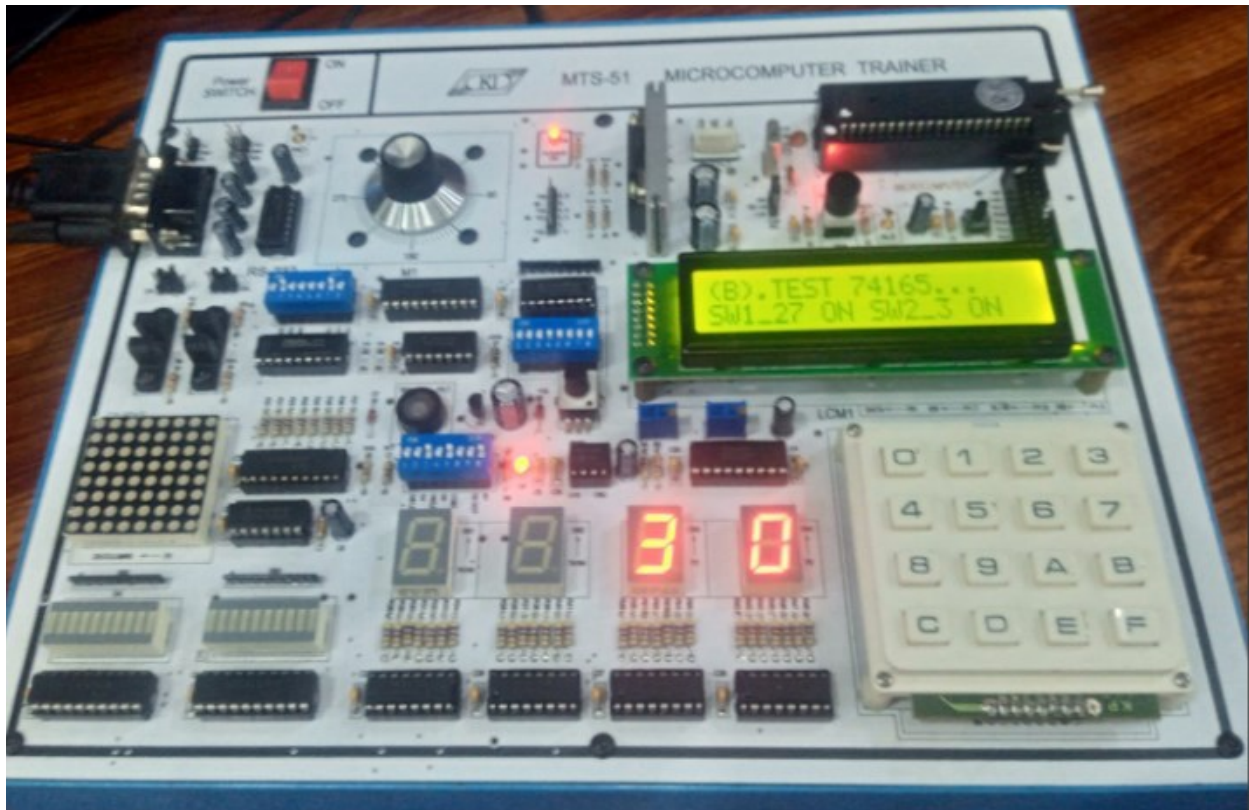
OUTPUT:



TASK#11:

Demo example 1 for SERIAL COMMUNICATION on DS4 of 2ND trainees is demonstrated by configuring switches as in the previous lab. The steps for successful run are documented, along with a snapshot of the final output.

OUTPUT:



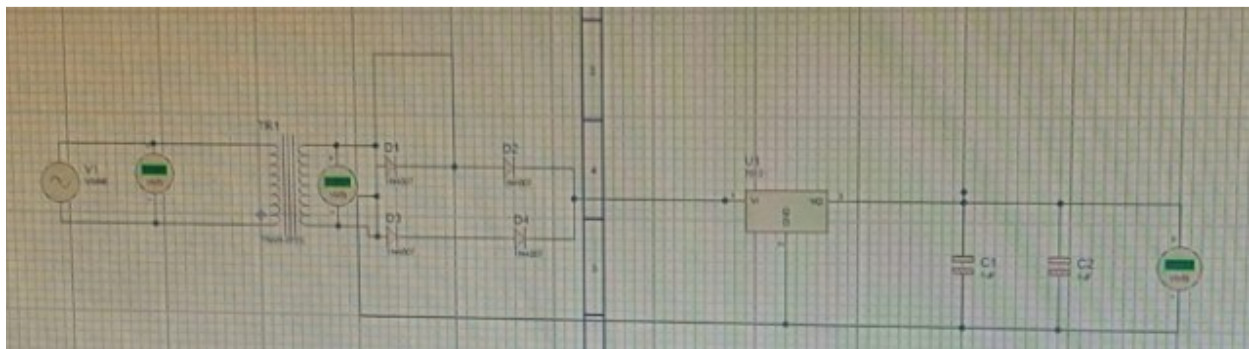
Experiment #4

Introduction to Proteus software with examples.

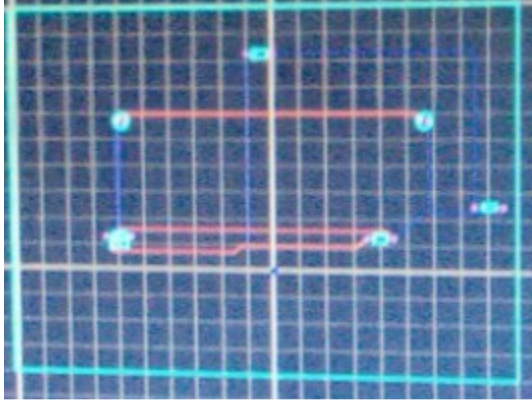
TASK#1:

In this task I had to make schematic diagram by using the proteus software and also convert the schematic diagram into pcb diagram.

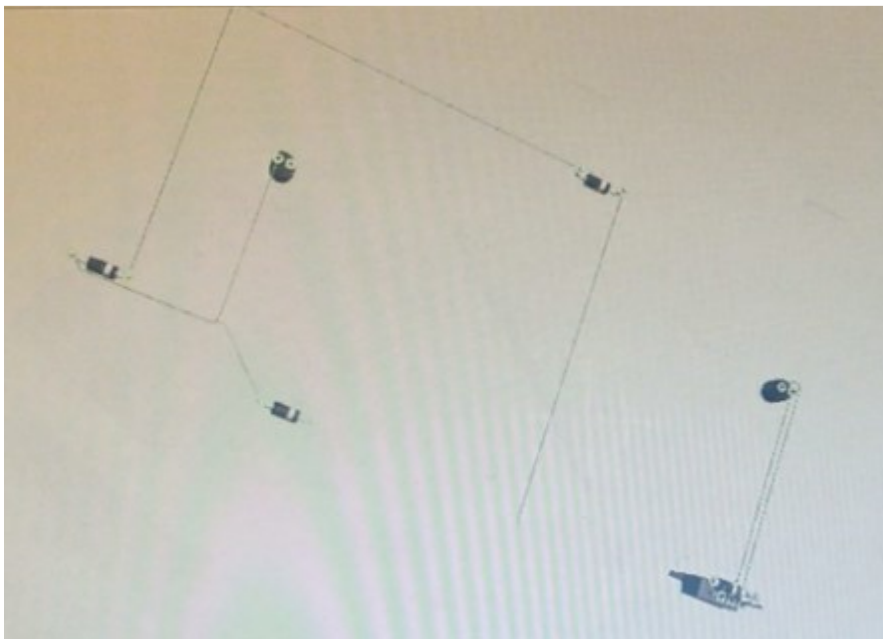
PROTEUS DIAGRAM:



PCB DIAGRAM:



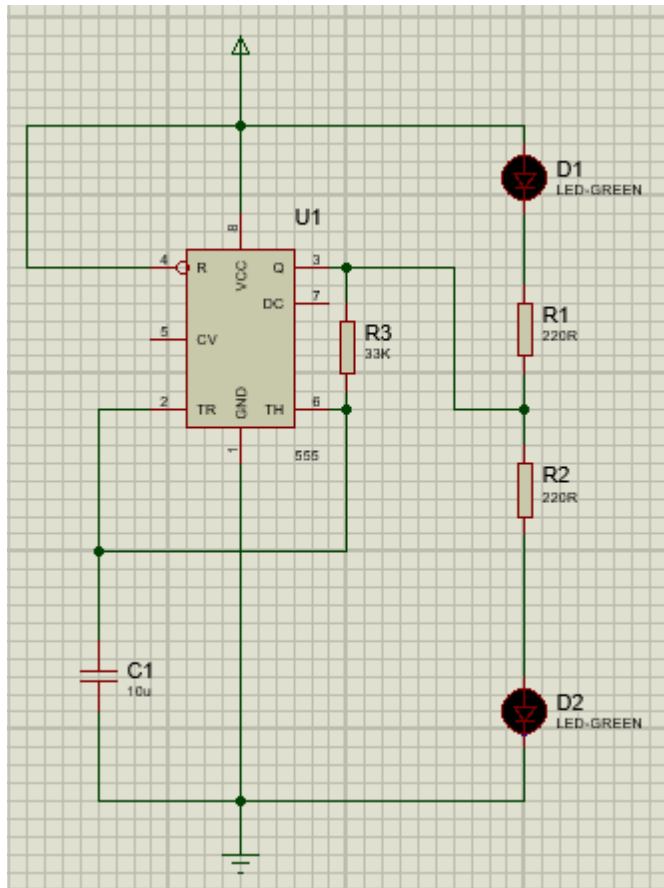
OUTPUT:



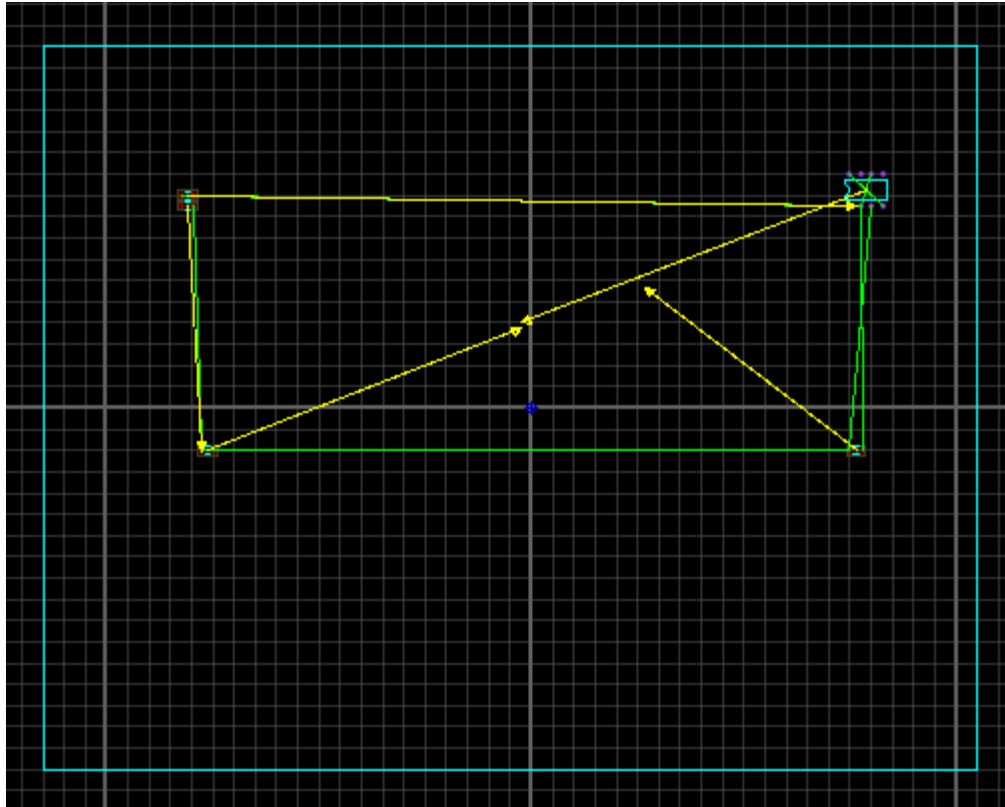
TASK#2:

In this task I had to make schematic diagram by using the proteus software and also convert the schematic diagram into pcb diagram. Show the output by flashing two LEDs with 555 Timer in Proteus ISIS by designing a circuit as well PCB layout shown in below figure.

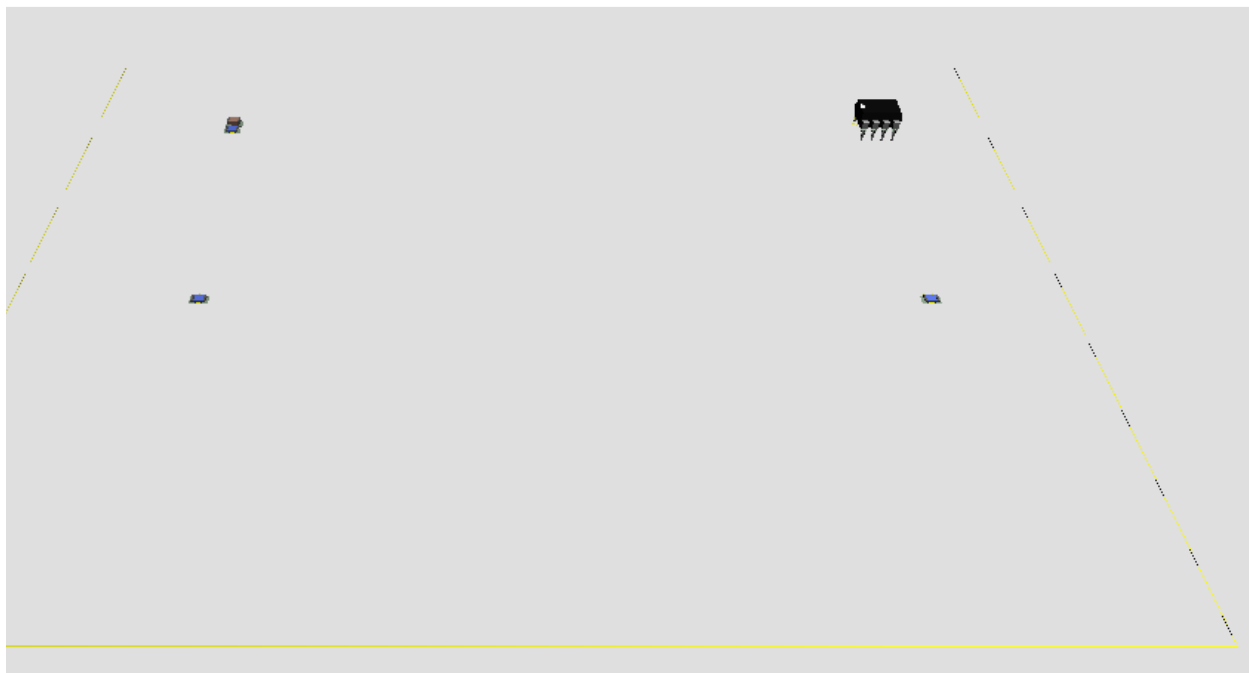
PROTEUS DIAGRAM:



PCB DIAGRAM:



OUTPUT:



Experiment #5

Led Pattern generation using EdSim51 Simulator

TASK#1:

Write a program which act as up and down counter from 0 to 9 and 9 to 0 using the 8-LEDs.

CODE:

```
ORG 0000H      ; Start at address 0
MOV P1, #00H   ; Initialize Port 1 to 0 (all LEDs off)
MOV R0, #00H   ; Initialize counter value to 0
MOV R1, #09H   ; Set maximum count to 9
UP_COUNT:
    MOV A, R0   ; Move the counter value to the accumulator
    MOV P1, A   ; Display the counter value on LEDs
    ACALL DELAY ; Call delay subroutine
    INC R0      ; Increment counter
    CJNE R0, R1, UP_COUNT ; If R0 != 9, continue counting up
    SJMP DOWN_COUNT ; Jump to DOWN_COUNT when reaching 9
DOWN_COUNT:
    MOV A, R0   ; Move the counter value to the accumulator
    MOV P1, A   ; Display the counter value on LEDs
    ACALL DELAY ; Call delay subroutine
    DEC R0      ; Decrement counter
    CJNE R0, #00H, DOWN_COUNT ; If R0 != 0, continue counting
down
    SJMP UP_COUNT ; Jump back to UP_COUNT when reaching 0
DELAY:          ; Delay subroutine for LED visibility
```

MOV R2, #250 ; Load R2 with 250 (outer loop counter)

D1: MOV R3, #250 ; Load R3 with 250 (inner loop counter)

D2: DJNZ R3, D2 ; Decrement R3 until it reaches 0

DJNZ R2, D1 ; Decrement R2 until it reaches 0

RET ; Return from delay

END ; End of program

OUTPUT:

The screenshot displays the Proteus ISIS simulation environment. The top window shows the assembly code for a delay routine. The code includes comments in Chinese and assembly instructions for loading registers, decrementing, and jumping. The bottom window shows the hardware components, including a DAC, a scope, and a 7-segment display.

Assembly Code:

```

UP_COUNT:
0007| MOV A, R0 ; Move the
0008| MOV P1, A ; Display t
000A| ACALL DELAY ; Call del
000C| INC R0 ; Increment
000D| SJMP DOWN_COUNT ; Jump to I

DOWN_COUNT:
000F| MOV A, R0 ; Move the
0010| MOV P1, A ; Display t
0012| ACALL DELAY ; Call del
0014| DEC R0 ; Decrement
0015| CJNE R0, #00H, DOWN_COUNT ;
0018| SJMP UP_COUNT ; Jump back

DELAY: ; Delay sub
001A| MOV R2, #250 ; Load R2 v
001C| D1: MOV R3, #250 ; Load R3 v
001E| D2: DJNZ R3, D2 ; Decrement
  
```

Hardware Components:

- DI, LD:** Input and Load buttons.
- 7-segment display:** Shows the value 8.8.8.8.
- Scope:** Displays the output of the DAC.
- AND Gate Disabled:** A status indicator.
- Key Bounce Disabled:** A status indicator.
- Standard:** A dropdown menu.
- 8-bit UART @ 4800 Baud:** A serial communication interface.
- Rx Reset, Tx Send:** Buttons for serial communication.

TASK#2:

Write a program which turn on the rightmost led by pressing the leftmost switch and so on.

CODE:

ORG 0000H ; Start of program

MOV P1, #0FFH ; Set all LEDs off initially (P1 as output port, active low LEDs)

MOV P3, #0FFH ; Set all switches off initially (P3 as input port)

MAIN:

MOV A, P3 ; Read the switches on Port 3

CPL A ; Complement the bits to match active-low logic for switches

MOV P1, #0FFH ; Reset LEDs to all OFF (high)

JNB ACC.0, TURN_ON_LED7 ; If SW0 is pressed, turn on LED7 (rightmost)

JNB ACC.1, TURN_ON_LED6 ; If SW1 is pressed, turn on LED6

JNB ACC.2, TURN_ON_LED5 ; If SW2 is pressed, turn on LED5

JNB ACC.3, TURN_ON_LED4 ; If SW3 is pressed, turn on LED4

JNB ACC.4, TURN_ON_LED3 ; If SW4 is pressed, turn on LED3

JNB ACC.5, TURN_ON_LED2 ; If SW5 is pressed, turn on LED2

JNB ACC.6, TURN_ON_LED1 ; If SW6 is pressed, turn on LED1

JNB ACC.7, TURN_ON_LED0 ; If SW7 is pressed, turn on LED0 (leftmost)

SJMP MAIN ; Repeat the loop

TURN_ON_LED7:

MOV P1, #7FH ; 0111 1111, turns on LED7

```
SJMP MAIN
TURN_ON_LED6:
    MOV P1, #BFH ; 1011 1111, turns on LED6
    SJMP MAIN
TURN_ON_LED5:
    MOV P1, #DFH ; 1101 1111, turns on LED5
    SJMP MAIN
TURN_ON_LED4:
    MOV P1, #EFH ; 1110 1111, turns on LED4
    SJMP MAIN
TURN_ON_LED3:
    MOV P1, #F7H ; 1111 0111, turns on LED3
    SJMP MAIN
TURN_ON_LED2:
    MOV P1, #FBH ; 1111 1011, turns on LED2
    SJMP MAIN
TURN_ON_LED1:
    MOV P1, #FDH ; 1111 1101, turns on LED1
    SJMP MAIN
TURN_ON_LED0:
    MOV P1, #FEH ; 1111 1110, turns on LED0
    SJMP MAIN
END
```

OUTPUT:

The screenshot displays the ED SIM5 IDE interface, which is used for simulating 8051 microcontroller programs. The top section shows the assembly code being executed, with the current instruction being `SJMP 0F5H` at address `0x003E`. The code includes a `main` function that initializes `P1` and `R0`, and a `loop` that checks for a key press on `P3.0` and increments `R0` if pressed. The hardware simulation at the bottom shows a green PCB with various components: a DIP switch, a 7-segment display (showing '8888'), a DAC, and a UART interface. The registers window shows the `PC` (Program Counter) at `0x0035` and the `PSW` (Program Status Word) at `0x0000`. The data memory window shows the contents of the 256 bytes of RAM.

TASK#3:

Write a program code which shows the number of key pressed on LED bar.

CODE:

\$NOLIST

\$MODLP51

\$LIST

```

org 0000H
    ljmp main
org 0030H
key_isr:
    inc R0      ; Increment key press counter in R0
    mov P1, R0  ; Display counter value on LED bar
    reti
ORG 0100H
main:
    mov SP, #7FH ; Initialize stack pointer
    mov R0, #00H ; Initialize key press counter to 0
    mov IE, #81H ; Enable external interrupt 0 (P3.2)
    mov TCON, #01H ; Enable falling edge detection on P3.2
    ljmp $
END

```

OUTPUT:

The screenshot displays the Keil uVision IDE interface. The top panel shows the assembly code for an 8051 microcontroller. The code includes a main routine that initializes the stack pointer (SP) to 0x7FH, sets up the key input (R0), and enables the external interrupt (IE). The key_isr routine increments the key counter (R0) and displays it on the 8-bit LED display. The hardware interface at the bottom shows the 8051 microcontroller with its registers (R0-R7, ACC, PSW, IP, IE, PCON, DPH, DPL, SP) and the 8-bit LED display. The LED display shows the value 0x00. The hardware interface also includes a DAC, a scope, and a UART interface.

Assembly Code:

```

org 0000H
ljmp main

org 0030H
key_isr:
0030H inc R0 ; Increment key p
0031H mov P1, R0 ; Display counter
0033H reti

ORG 0100H
main:
0100H mov SP, #7FH ; Initialize stack
0103H mov R0, #00H ; Initialize key
0105H mov IE, #81H ; Enable external
0108H mov TCON, #01H ; Enable falling
010BH RET
END

```

Register Values:

R7	0x00	B	0x00
R6	0x00	ACC	0x00
R5	0x00	PSW	0x00
R4	0x00	IP	0x00
R3	0x8E	IE	0x81
R2	0xF7	PCON	0x00
R1	0x09	DPH	0x00
R0	0x00	DPL	0x00
		SP	0x7F

PC: 0x010B

Data Memory:

addr	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00	00	00	09	F7	8E	00	00	00	00	00	00	00	00	00	00	00
10	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
20	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
30	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
50	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
60	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
70	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Hardware Interface:

- DI: 0, LD: 0
- AND Gate Disabled
- Key Bounce Disabled
- Standard
- UART: 8-bit UART @ 4800 Baud
- Rx: No Parity
- Tx: No Parity
- 0.0 V output
- Scope
- DAC
- BF: 0, AC: 0x00, IR: 0x00, DR: 0x00
- LED Display: 0000

Experiment #6

Introduction to Keil Debugger and simulator with Led Display Control using MTS-51.

TASK#1:

Write a program which Sequentially and repeatedly turn on/off the 8 LEDs from the leftmost LED to the rightmost LED. The ON time of each LED is 0.1 seconds.

CODE:

ORG 0000H ; Start at address 0

MOV P1, #00H ; Clear all LEDs initially (assuming LEDs are connected to Port 1)

MOV R0, #08H ; Counter for 8 LEDs

MAIN:

MOV A, #01H ; Initialize with leftmost LED (binary 00000001)

MOV R1, #08H ; Load the loop count for 8 LEDs

LOOP:

MOV P1, A ; Output current LED state to Port 1

ACALL DELAY ; Delay for 0.1 seconds

RL A ; Rotate left to shift the bit to the next LED

DJNZ R1, LOOP ; Decrement R1, repeat for 8 LEDs

SJMP MAIN ; Jump back to start for continuous operation

DELAY: ; 0.1-second delay subroutine (approximate)

MOV R2, #250

DELAY_LOOP1:

MOV R3, #250

DELAY_LOOP2:

DJNZ R3, DELAY_LOOP2

DJNZ R2, DELAY_LOOP1

RET

END

OUTPUT:

Build Output

```
creating hex file from "..\Objects\turnonoffthe8leds"...  
"..\Objects\turnonoffthe8leds" - 0 Error(s), 3 Warning(s).  
Build Time Elapsed: 00:00:01
```

TASK#2:

Write a program which Sequentially turn on/off the 8 LEDs from the leftmost LED to the rightmost LED. When the rightmost LED is reached, the sequence is inverted. The ON time of each LED is 0.1 seconds.

CODE:

ORG 0000H ; Start of code memory

START:

MOV P1, #01H ; Initialize the leftmost LED (bit 0 of Port 1)

MOV R0, #01H ; Set up the direction for the sequence (left to right)

LOOP:

MOV P1, A ; Update the Port 1 to the value of A register

ACALL DELAY ; Call the delay subroutine

MOV A, P1 ; Load the current state of LEDs into A

RLC A ; Rotate left through carry (move to next LED in sequence)

MOV P1, A ; Update the Port 1 with the new value

ACALL DELAY ; Call the delay subroutine

JNB P1.7, LOOP ; Jump back to LOOP if the rightmost LED is not reached

; Invert the direction (right to left)

MOV P1, #80H ; Set the rightmost LED

MOV R0, #80H ; Set up the direction for the sequence (right to left)

INVERT_LOOP:

MOV P1, A ; Update Port 1 with the current value

ACALL DELAY ; Call the delay subroutine

MOV A, P1 ; Load the current state of LEDs into A

RRC A ; Rotate right through carry (move to previous LED in sequence)

MOV P1, A ; Update the Port 1 with the new value

ACALL DELAY ; Call the delay subroutine

JNB P1.0, INVERT_LOOP ; Jump back to INVERT_LOOP if the leftmost LED is not reached

; Repeat the sequence

SJMP LOOP ; Jump back to the start of the loop

DELAY:

MOV R1, #200 ; Outer loop count

DELAY1:

MOV R2, #250 ; Inner loop count

DELAY2:

NOP ; No Operation (used for delay)

NOP

NOP

NOP

DJNZ R2, DELAY2 ; Decrement inner loop counter

DJNZ R1, DELAY1 ; Decrement outer loop counter

RET ; Return from the delay subroutine

END

OUTPUT:

Build Output

```
Program Size: data=9.0 xdata=0 code=68  
".\Objects\LED" - 0 Error(s), 3 Warning(s).  
Build Time Elapsed: 00:00:00
```

Build Output

```
creating hex file from ".\Objects\LED"..  
".\Objects\LED" - 0 Error(s), 3 Warning(s).  
Build Time Elapsed: 00:00:01
```

TASK#3:

Write a program which Sequentially turn on/off the two LEDs from two ends towards the middle. When the middle reached, the sequence is inverted. The ON time of each LED is 1 seconds.

CODE:

ORG 0000H ; Start of program memory

START:

MOV P1, #0x01 ; Turn on the leftmost LED (P1.0)

ACALL DELAY ; Wait for 1 second

MOV P1, #0x00 ; Turn off the leftmost LED (P1.0)

ACALL DELAY ; Wait for brief time before next action

MOV P1, #0x80 ; Turn on the rightmost LED (P1.7)

ACALL DELAY ; Wait for 1 second

MOV P1, #0x00 ; Turn off the rightmost LED (P1.7)

ACALL DELAY ; Wait for brief time before next action

MOV R0, #0 ; Initialize R0 as a counter for middle

MOV R1, #7 ; Set R1 to 7 (for P1.7)

; Move towards the middle from both ends

LOOP:

MOV A, P1 ; Check the current status of LEDs

INC R0 ; Increment counter from left side (P1.0)

MOV P1, #0x01 << R0 ; Turn on the next LED from left end

ACALL DELAY ; Wait for 1 second

DEC R1 ; Decrement counter from right side (P1.7)

MOV P1, #0x80 >> R1 ; Turn on the next LED from the right end

ACALL DELAY ; Wait for 1 second

; Check if the middle is reached

CJNE R0, R1, LOOP ; If the left end and right end don't meet, continue

SJMP INVERT_SEQUENCE ; If the middle is reached, invert sequence

; Invert the sequence and go back to the end

INVERT_SEQUENCE:

MOV R0, #7 ; Set R0 back to the rightmost end (P1.7)

MOV R1, #0 ; Set R1 back to the leftmost end (P1.0)

REVERSE_LOOP:

MOV A, P1 ; Check current LED state

DEC R0 ; Move towards left from right end

MOV P1, #0x01 << R0 ; Turn on next LED from right to left

```

ACALL DELAY      ; Wait for 1 second
INC R1           ; Move towards right from left end
MOV P1, #0x80 >> R1 ; Turn on next LED from left to right
ACALL DELAY      ; Wait for 1 second
; Check if the middle is reached again
CJNE R0, R1, REVERSE_LOOP ; Continue until the middle is
reached
SJMP START      ; Repeat the cycle
; Delay subroutine (approx. 1 second)
DELAY:
    MOV R2, #250 ; Outer loop count
DELAY1:
    MOV R3, #250 ; Inner loop count
DELAY2:
    NOP          ; No operation (just for delay)
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP

```

NOP

NOP

NOP

NOP

NOP

NOP

NOP

NOP

NOP

DJNZ R3, DELAY2 ; Decrement and loop if not zero

DJNZ R2, DELAY1 ; Decrement and loop if not zero

RET

END

OUTPUT:

Build Output

```
Program Size: data=9.0 xdata=0 code=53  
".\Objects\INVERTEDLED" - 0 Error(s), 3 Warning(s).  
Build Time Elapsed: 00:00:00
```

<

Build Output

```
creating hex file from ".\Objects\INVERTEDLED"..  
".\Objects\INVERTEDLED" - 0 Error(s), 3 Warning(s).  
Build Time Elapsed: 00:00:00
```

<

TASK#4:

Write a program which toggle between the even led and odd led having the delay of 2 sec.

CODE:

; Program to toggle between even and odd LEDs on an 8051 microcontroller

; using Keil Assembly Language

ORG 0000H ; Origin, start of the program

START: ; Start label

MOV P1, #0 ; Initialize Port 1 to 0 (all LEDs off)

MAIN_LOOP: ; Main loop label

MOV A, P1 ; Move current state of Port 1 to Accumulator A

CPL A ; Complement the accumulator (toggle the state)

MOV P1, A ; Output the toggled state back to Port 1

ACALL DELAY ; Call delay subroutine

SJMP MAIN_LOOP ; Jump back to main loop

DELAY: ; Delay subroutine

MOV R0, #20 ; Load R0 with a value for delay

DELAY_LOOP:

NOP ; No operation (do nothing)

DJNZ R0, DELAY_LOOP ; Decrement R0 and repeat if not zero

RET ; Return from subroutine

END ; End of program

OUTPUT:

Build Output

```
Program Size: data=9.0 xdata=0 code=33
".\Objects\TOGGLELED" - 0 Error(s), 3 Warning(s).
Build Time Elapsed: 00:00:00
```

Build Output

```
creating hex file from ".\Objects\TOGGLELED"...\n".\Objects\TOGGLELED" - 0 Error(s), 3 Warning(s).\nBuild Time Elapsed: 00:00:00
```

TASK#5:

Write a program which Uses the technique of table lookup to show the LED patterns resided in memory. Each pattern stays on 0.1 seconds.

CODE:

; 8051 Assembly Code for LED Patterns using Table Lookup

; This program demonstrates LED patterns stored in memory.

; Each pattern stays on for 0.1 seconds.

ORG 0000H ; Origin, start of program memory

; Define the LED patterns in a lookup table

PATTERNS: DB 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80 ; 8
patterns for LEDs

; Define constants

DELAY_COUNT EQU 250 ; Adjust this value to achieve approximately
0.1 seconds delay

; Main program starts here

START:

MOV DPTR, #PATTERNS ; Point to the start of the patterns table

MOV R0, #08 ; Set R0 to number of patterns (8)

MAIN_LOOP:

CLR A ; Clear accumulator

MOVC A, @A+DPTR ; Load pattern from lookup table into accumulator

MOV P1, A ; Output the pattern to Port 1 (LEDs connected here)

ACALL DELAY ; Call delay subroutine

INC DPTR ; Move to the next pattern in the table

DJNZ R0, MAIN_LOOP ; Decrement R0 and repeat if not zero

SJMP START ; Repeat the main loop indefinitely

DELAY:

MOV R1, #DELAY_COUNT ; Load delay count into R1

DELAY_LOOP:

DJNZ R1, DELAY_LOOP ; Decrement R1 until it reaches zero

RET ; Return from delay subroutine

END ; End of program

OUTPUT:

Build Output

```
Program Size: data=9.0 xdata=0 code=44
".\Objects\LOOKUPLED" - 0 Error(s), 3 Warning(s).
Build Time Elapsed: 00:00:00
```

Build Output

```
creating hex file from ".\Objects\LOOKUPLED"...
".\Objects\LOOKUPLED" - 0 Error(s), 3 Warning(s).
Build Time Elapsed: 00:00:01
```

TASK#6:

Write a program which Turn on LEDs according to the following cases and repeat each

case 8 times.

- a. Case 1: Turn on LEDs from left to right.
- b. Case 1: Turn on LEDs from right to left.
- c. Blink two sets (left and right sides) of LEDs alternately.
- d. Blink all of 8 LEDs.

OUTPUT:

```
ORG 0000H
START:
MOV P1, #00H
MOV R0, #08H
CASE1_LEFT:
MOV R1, #00H
LEFT_LOOP:
MOV P1, #00H
SETB P1.0
ACALL DELAY
INC R1
CJNE R1, #08H, LEFT_LOOP
ACALL DELAY
CASE2_RIGHT:
MOV R1, #07H
RIGHT_LOOP:
MOV P1, #00H
SETB P1.7
ACALL DELAY
DJNZ R1, RIGHT_LOOP
ACALL DELAY
CASE3_BLINK_SETS:

BLINK_LOOP:
MOV P1, #F0H
ACALL DELAY
```

```

MOV P1, #0FH
ACALL DELAY
DJNZ R0, BLINK_LOOP
CASE4_BLINK_ALL:
BLINK_ALL_LOOP:
MOV P1, #00H
ACALL DELAY
MOV P1, #FFH
ACALL DELAY
DJNZ R0, BLINK_ALL_LOOP
JMP START
DELAY:
MOV R2, #255
DELAY_OUTER:
MOV R3, #255
DELAY_INNER:
DJNZ R3, DELAY_INNER
DJNZ R2, DELAY_OUTER
RET
END

```

OUTPUT:

Build Output

```

Program Size: data=9.0 xdata=0 code=81
".\Objects\8 LEDs" - 0 Error(s), 3 Warning(s).
Build Time Elapsed: 00:00:02

```

Build Output

```

creating hex file from ".\Objects\8 LEDs"...
".\Objects\8 LEDs" - 0 Error(s), 3 Warning(s).
Build Time Elapsed: 00:00:00

```

Experiment #7

Demonstration and practice of Dot Matrix LED Control and display different patterns.

TASK#1:

Display the example in the manual and analyze the output.

CODE:

ORG 0000H

MOV P1, #0FFH

MOV P2, #00H

A_PATTERN: DB 0x18, 0x3C, 0x66, 0xC3, 0xFF, 0xC3, 0xC3, 0xC3

MAIN: ; Main loop to continuously display the letter

MOV R0, #A_PATTERN

DISPLAY:

MOV R1, #08H

DISPLAY_ROW:

MOV A, @R0

MOV P1, A

MOV A, R1

CPL A

MOV P2, A

ACALL DELAY

INC R0

MOV A, R1

RL A

MOV R1, A

DJNZ R1, DISPLAY_ROW

SJMP MAIN

DELAY:

MOV R2, #255

DELAY_LOOP1:

DJNZ R2, DELAY_LOOP1

MOV R2, #255

DELAY_LOOP2:

DJNZ R2, DELAY_LOOP2

RET

END

OUTPUT:

Build Output

```
creating hex file from ".\Objects\dotmartixled"...  
".\Objects\dotmartixled" - 0 Error(s), 3 Warning(s).  
Build Time Elapsed: 00:00:01
```

TASK#2:

Write a program which display the alphabet A on Dot Matrix LED.

CODE:

ORG 0000H

PATTERN: DB 3CH, 66H, 66H, 7EH, 66H, 66H, 66H, 00H

ROW_SELECT: DB 0FEH, 0FDH, 0FBH, 0F7H, 0EFH, 0DFH, 0BFH, 0FFH

MOV DPTR, #PATTERN

MOV P0, #0FFH

MOV P2, #0FFH ; Clear Port 2 (connected to rows)

MAIN_LOOP:

```

    MOV R0, #08    ; Loop counter for 8 rows
DISPLAY_LOOP:
    MOVX A, @DPTR   ; Load row data from pattern
    MOV P2, A       ; Output row data to Port 2 (74LS245 for rows)
    MOV A, R0
    DEC A           ; Convert R0 to match row index (0-7)
    MOVC A, @A+ROW_SELECT ; Get row selection value from
ROW_SELECT
    MOV P0, A       ; Output to Port 0 (ULN2803 for columns)
    ACALL DELAY     ; Call delay for stable display
    INC DPTR        ; Move to next row in the pattern
    DJNZ R0, DISPLAY_LOOP ; Repeat for all rows
    MOV DPTR, #PATTERN ; Reset DPTR to start of pattern
    SJMP MAIN_LOOP  ; Repeat the main loop indefinitely
; Simple delay subroutine
DELAY:
    MOV R1, #0FFH
    MOV R2, #0FFH
DELAY_LOOP:
    DJNZ R2, DELAY_LOOP
    DJNZ R1, DELAY_LOOP
    RET
END

```


OUTPUT:

Build Output

```
creating hex file from ".\Objects\dotmartixled"...  
".\Objects\dotmartixled" - 0 Error(s), 3 Warning(s).  
Build Time Elapsed: 00:00:00
```

TASK#3:

```
ORG 0000H  
MOV DPTR, #L_A  
CALL DISPLAY  
MOV DPTR, #L_B  
CALL DISPLAY  
MOV DPTR, #L_C  
CALL DISPLAY  
SJMP $  
DISPLAY:  
MOV R0, #8  
DISPLAY_LOOP:  
MOVB A, @DPTR  
MOV P1, A  
ACALL DELAY  
INC DPTR  
DJNZ R0, DISPLAY_LOOP  
CALL DELAY_500MS  
RET  
L_A: DB 0x3C, 0x66, 0xC3, 0xFF, 0xC3, 0xC3, 0xC3, 0x00  
L_B: DB 0xFC, 0x66, 0x66, 0xFC, 0x66, 0x66, 0xFC, 0x00  
L_C: DB 0x3C, 0x66, 0xC0, 0xC0, 0xC0, 0x66, 0x3C, 0x00  
DELAY_500MS:  
MOV R1, #50 D1: MOV R2, #255  
D2: MOV R3, #255  
DJNZ R3, D2  
DJNZ R2, D1  
  
DJNZ R1, DELAY_500MS
```

```
RET
DELAY:
MOV R4, #100
D3: DJNZ R4, D3
RET
END
```

OUTPUT:

Build Output

```
Program Size: data=9.0 xdata=0 code=72
".\Objects\dotmartixledaplahabet" - 0 Error(s), 3 Warning(s).
Build Time Elapsed: 00:00:00
```

Build Output

```
creating hex file from ".\Objects\dotmartixledaplahabet"...
".\Objects\dotmartixledaplahabet" - 0 Error(s), 3 Warning(s).
Build Time Elapsed: 00:00:01
```